From Algorithm to RTL: A Comparative Study of Compiler-Driven and LLM-Driven FPGA Design Flows

Md Mahfuzul Haque Gazi[1], Rickard Ewetz[2], Hao Zheng[3], Md Rubel Ahmed[1]
[1]Louisiana Tech University, [2]University of Florida, [3]University of Central Florida
Email: {mha059, mahmed}@latech.edu[1], rewetz@ufl.edu[2], hao.zheng@ucf.edu[3]

***Abstract*—Modern Large Language Models (LLMs) can generate SystemVerilog directly from high-level PyTorch descriptions with minimal effort, allowing for rapid push-button hardware prototyping. Compiler-based hardware generation flows built around dialects and frameworks such as MLIR and CIRCT require substantial upfront engineering investment to develop, integrate, and maintain. This paper seeks to quantitatively and qualitatively evaluate whether low-effort, LLM-driven RTL generation has the potential to realistically substitute these engineering-heavy compiler toolchains in practical design settings. The investigation systematically compares LLM-generated versus compiler-generated hardware artifacts across a range of ML kernels, spanning simple components like activation functions to more structured workloads such as FFNN layers and attention blocks. Our results show that LLM-based flows are competitive for simple kernels and significantly improve productivity, but they struggle to scale to complex workloads where performance optimization is critical. Compiler-driven flows consistently outperform LLM-generated RTL on complex kernels by enabling effective scheduling, data and operation fusion, and memory optimization, achieving up to 1.5× higher throughput and 50% lower resource usage. We further analyze whether this performance gap arises primarily from increasing design complexity and context-length limitations, or from inherent shortcomings in LLM-generated RTL that restrict kernel-level optimizations required for high throughput. Our findings indicate that LLMs support rapid design exploration, whereas compiler = infrastructures continue to dominate predictable, scalable, and high-performance hardware generation.**

I. Introduction

Rapid progress in compiler infrastructures for hardware design has been driven by the growing demand for efficient accelerators for machine-learning workloads [1], [2]. This demand has propelled the development of compiler frameworks such as Allo [3], MLIR [4], CIRCT [5], and Calyx [6], which enable systematic lowering from high-level PyTorch programs to synthesizable SystemVerilog and provide structured opportunities for optimization through scheduling, pipelining, and memory transformations. These compiler-driven approaches have demonstrated strong potential for producing high-performance hardware, but they often require substantial engineering effort, including complex environment setup and the design of intermediate representations and compiler passes [4], [5].

At the same time, commercial LLMs have emerged as a new paradigm for RTL generation [7], [8]. Several studies have investigated enhancing RTL generation capabilities through customized models and training paradigms [9]. These LLM-based tools can generate SystemVerilog directly from Python- or PyTorch-level descriptions with minimal user intervention, enabling rapid push-button RTL prototyping [9], [10]. This ease of use has led to growing interest in whether LLM-driven hardware generation can reduce reliance on complex compiler toolchains [7], [9].

Despite their convenience, the performance competitiveness of LLM-generated RTL remains underexplored. Generating functionally correct code is only the first step in hardware design; real-world accelerators must satisfy stringent performance, area, and timing constraints [1], [2]. Whereas compiler-based flows apply deterministic analyses and optimization passes over explicit intermediate representations [4], [5], LLMs rely on probabilistic pattern matching during code generation [7], [8]. This contrast raises a central question: *does the substantial engineering investment required to build and maintain compiler toolchains still provide sufficient value when LLMs can generate RTL almost instantly, and under what conditions do the performance, optimization capability, and reliability of compiler-generated designs justify this cost?*

In this paper, we present an empirical comparison of compiler-driven and commercial LLM-driven SystemVerilog generation across a range of representative machine-learning kernels, including GEMM, feed-forward layers, attention blocks, and ReLU activations. Using a unified synthesis and evaluation flow, we evaluate both approaches in terms of throughput, maximum achievable clock frequency, and hardware resource utilization. The key contributions of the paper are, as follows:

- We introduce a controlled and reproducible methodology for comparing compiler-generated and LLM-generated SystemVerilog derived from identical high-level kernel specifications.
- We report detailed performance, area, and efficiency metrics for representative ML workloads.
- We analyze the root causes of performance disparities between the two approaches, identifying architectural and optimization gaps that current LLM-based generation methods struggle to address.

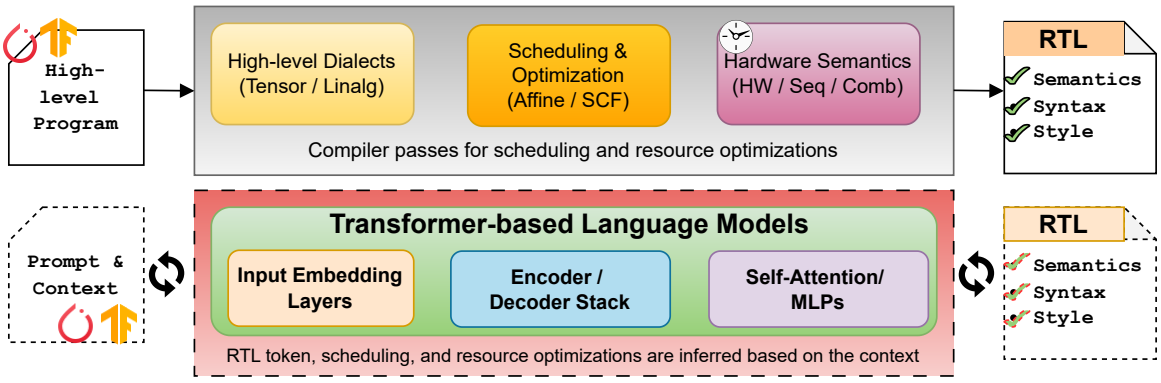

Fig. 1. Two RTL generation paths evaluated in this work. The compiler-driven flow (top) resolves scheduling, optimization, and hardware semantics through explicit, staged IR lowering, enabling deterministic and analyzable RTL generation. In contrast, the LLM-assisted flow (bottom) produces SystemVerilog through a compact generative pipeline, where scheduling, resource allocation, and structural decisions are inferred implicitly from the prompt and context rather than exposed as explicit intermediate stages. While LLM generation can be repeatable under fixed decoding settings, the resulting RTL structure is not governed by an explicit scheduling or optimization model.

II. Background and Related Work

This section reviews prior work on RTL generation using compiler-driven and generative approaches from high-level algorithmic and Python-based descriptions, with a focus on synthesizable SystemVerilog. We highlight key differences between explicit, pass-driven compilation pipelines and generative RTL methods to provide context for the comparative evaluation in Fig. 1.

1) Compiler-Driven SystemVerilog Generation: Compiler-driven hardware generation follows a staged lowering process in which a program is progressively transformed from an abstract computation into cycle-accurate, synthesizable SystemVerilog [11]–[16]. Each stage reduces abstraction in a controlled manner, while compiler passes explicitly resolve optimization, scheduling, and structural decisions. Consider the following high-level intent for a 1D tensor operation:

```
y = relu(x + b)
```

We use this example to illustrate how progressive lowering exposes hardware-relevant structure. Lowering is expressed through a sequence of intermediate representations (IR) known as *dialects*. A dialect defines a set of (1) operations, (2) types, and (3) invariants appropriate to a particular level of abstraction.

For the example tensor operation, it begins with high-level tensor and algebraic representations, expressed using dialects such as `torch/tf`, `tosa`, and `linalg`, where functional intent is preserved without explicit control flow, storage, or timing. These operations are then lowered into explicit iteration spaces using `linalg`, `affine`, and `scf`, exposing loop bounds and access patterns and enabling schedule-shaping transformations such as tiling, unrolling, fusion, and reordering. Bufferization subsequently converts tensor values into concrete storage, introducing explicit loads, stores, and scalar operations through dialects including `bufferization`, `memref`, `arith`, and `scf`, thereby fixing data placement and lifetimes. Hardware semantics are then introduced by mapping computation onto structural elements such as wires, registers, and control logic using CIRCT dialects `hw`, `comb`, and `seq`, making pipeline stages, state, and latency explicit. The fully resolved hardware IR is emitted as synthesizable SystemVerilog via the `sv` dialect, where the focus is faithful code generation as follows.

```
rtl.module @relu_add_sv(%clk: i1, %rst: i1,
                        %x: f32, %b: f32) {
  %y = sv.reg : !rtl.inout<f32>
  sv.alwaysff(posedge %clk) {
    %zf = sv.constant 0.0 : f32
    %sum = sv.addf %x, %b : f32
    %relu = sv.maxf %sum, %zf : f32
    sv.if (%rst) {
      sv.assign %y, %zf : f32
    } else {
      sv.assign %y, %relu : f32
    }
  }
}
```

2) LLM-Driven SystemVerilog Generation: As illustrated in Fig. 1 (bottom), LLM-driven SystemVerilog generation produces RTL through a compact, transformer-based generative pipeline [17], [18]. Given a prompt and contextual information, the model maps text into token embeddings and applies stacked encoder/decoder blocks with self-attention and MLP layers to perform autoregressive next-token prediction. RTL is emitted directly as a token sequence, without explicit intermediate representations for scheduling, optimization, or hardware semantics. Functional correctness emerges from learned statistical regularities in code corpora rather than from explicit semantic analysis [19]–[23].

```
Generate a synthesizable SystemVerilog module that computes
C = A * B for two 32-bit integers.

Requirements:
- One result per cycle after pipeline fill
- No vendor-specific IP
- Top module name: main
- Single clock input

Provide only the SystemVerilog code.
```

Because scheduling, resource allocation, and pipeline structure are not represented as explicit stages, these decisions are inferred implicitly from the prompt and context, as indicated by the dotted boundaries in Fig. 1.

III. Comparison Study Design

This work compares two approaches for generating hardware accelerators from machine learning kernels. The first is a compiler-driven flow, in which `Allo` programs are lowered and optimized through MLIR dialects before being emitted as synthesizable SystemVerilog. The second is an LLM-driven flow, where equivalent kernel descriptions and constraints are provided directly to large language models to generate RTL without an explicit compilation pipeline, as summarized in Fig. 2. We do not consider conventional C++-based HLS toolchains, as their manual restructuring and hardware-specific optimizations largely overlap with MLIR-based transformations and typically offer no advantage over compiler-driven MLIR lowering. The following sections discuss the comparative study design.

1) Benchmark Selection: To evaluate the effectiveness of compiler optimizations, we select five representative kernels spanning a wide range of computational complexity, from a simple arithmetic function to a bi-layer feed-forward neural network (FFNN) presented in Table I. These kernels were selected to capture both lightweight operations and more compute-intensive workloads commonly found in machine learning applications.

TABLE I
ALLO PROGRAM KERNELS AND MODELS USED IN THIS WORK.

Increment	Single 32-bit integer scalar input (A: int32); computes $A + 1$.
ReLU	Element-wise addition of two float32 tensors followed by ReLU, $\mathrm{ReLU}(x + y)$.
GELU	Computes GELU, $\mathrm{GELU}(x)$.
GEMM	Matrix multiplication of two 32×32 int32 matrices, $C = A \times B$.
FFNN	Two-layer feed-forward neural network with input dimension 64, hidden dimension 48, and output dimension 4, with an intermediate ReLU.

Among the kernels used, the increment kernel serves as a minimal baseline, while an integer 32×32 matrix multiplication kernel captures dense arithmetic computation. We also include ReLU and GELU activation kernels operating on $3 \times 10 \times 10$ float32 tensors, where ReLU performs an element-wise addition followed by activation. Finally, the FFNN kernel consists of a two-layer fully connected network with a 64×48 hidden layer and an output dimension of 4, representing a common neural network workload with both linear and nonlinear stages. Together, these benchmarks enable evaluation of compiler-driven hardware optimizations across simple and complex workloads.

2) RTL Generation: The initial benchmark implementations are written in Python. We used the Allo compiler to lower these programs into MLIR linalg dialect. The generated MLIR was then processed through a sequence of compiler passes, including transformations from linalg to affine, and further lowering into SCF and memref-based representations as shown

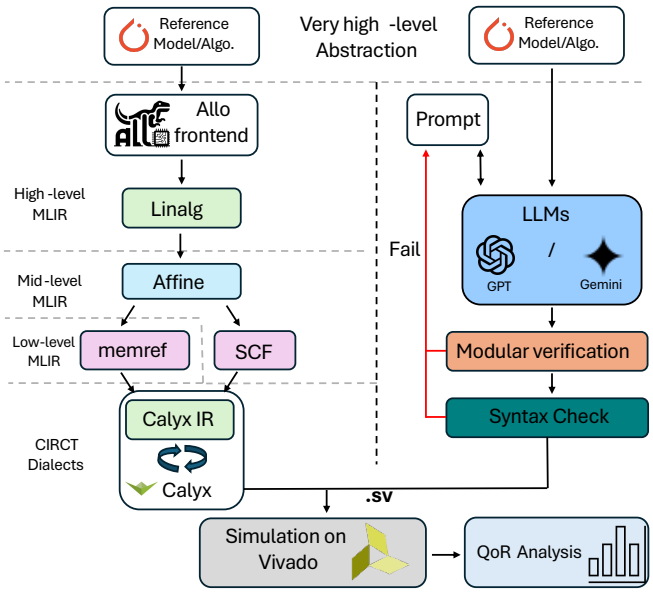

Fig. 2. Flows for compiler-driven and LLM-driven SV generation.

in Fig 2 (left) flow. Several additional passes were applied to improve compatibility with downstream hardware compilation. In particular, we removed global memory declarations and reduced multi-dimensional memrefs into one-dimensional buffers, making it suitable for CIRCT-based flows. After these transformations, CIRCT was used to convert the design into Calyx IR. Finally, the Calyx native compiler generated SystemVerilog from this intermediate representation. All experiments were conducted using Calyx compiler version 0.71, built from source to better support floating-point arithmetic.

For LLM-driven generation, we provided the python kernel and design constraints directly in the prompt. We evaluated GPT-5.2 [24] and Gemini 3 Pro [25], but both models often failed to produce correct floating-point implementations, especially for more complex kernels such as GEMM and FFNN. Code generation typically requires multiple prompt iterations, with Vivado used for syntax checking and error feedback. For larger models such as BERT, neither LLM was able to generate valid SystemVerilog after repeated attempts.

3) Simulation and QoR Extraction: All designs were synthesized and evaluated using Xilinx Vivado targeting the Zynq-7020 FPGA. We report standard FPGA implementation metrics, including maximum achievable clock frequency (Fmax), resource utilization in terms of LUTs, flip-flops (FFs), and DSP blocks, as well as estimated power consumption from Vivado reports.

All experiments were conducted on a Dell Pro Max T2 desktop workstation running Windows, equipped with 64 GB of RAM and an NVIDIA RTX ADA2000 GPU, with Ubuntu 22.04 LTS installed inside an Oracle VirtualBox virtual machine.

IV. Results & Analysis

We see significance performance gap between compiler passes compared to the LLM generation. On average across all

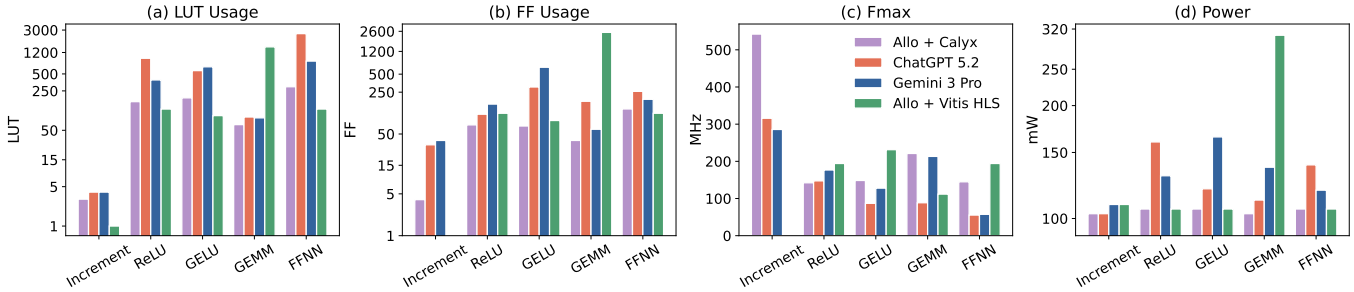

Fig. 3. Resource and performance comparison across kernels for Allo+Calyx, ChatGPT 5.2, and Gemini 3 Pro: (a) LUT usage, (b) FF usage, (c) maximum frequency (Fmax), and (d) power consumption.

TABLE II
OVERALL INPUT (IN) AND OUTPUT (OUT) TOKEN REQUIRED TO GENERATE SYNTHESIZABLE SYSTEMVERILOG

Kernel	**ChatGPT 5.2**		**Gemini 3 Pro**	
	IN	OUT	IN	OUT
Increment	220	180	210	95
ReLU	410	314	400	280
GELU	450	197	430	467
GEMM	620	540	600	200
FFNN	780	588	1050	255

kernels, the compiler-driven flow achieves about 1.7× higher Fmax compared to ChatGPT-generated RTL, and about 1.4× higher Fmax compared to Gemini. In terms of resource efficiency, the compiler uses approximately 75% fewer LUTs than ChatGPT and about 50% fewer LUTs than Gemini on average. These results show that while LLMs can occasionally match the compiler on simple kernels, compiler-generated designs consistently provide better performance and significantly lower area cost for more complex workloads.

The Allo + Calyx or MLIR + CIRCT compiler-driven flow consistently outperforms LLM-driven hardware generation because it is built on principled, semantics-preserving program transformations and composable scheduling abstractions rather than heuristic code synthesis. This flow enables progressive customization through well-defined compute, memory, and communication rewrites, ensuring correctness while aggressively optimizing resource usage and timing. Moreover, compiler passes explicitly target the kernel composition challenge by providing type-safe interfaces, memory-layout reasoning, and hierarchical dataflow analysis, including systematic FIFO sizing, which prevents the excessive buffering and control overhead often introduced in LLM-generated designs. In contrast, LLM-driven flows lack global optimization guarantees and frequently produce conservative pipelines and redundant logic, leading to higher LUT/FF counts, reduced Fmax, and increased power consumption. Besides, the compiler-driven flow is explainable and deterministic, producing stable RTL and consistent Vivado results across runs. In contrast, the LLM-driven flow is stochastic, and repeated generations can lead to noticeable differences in timing and resource usage.

Performance Gap in LLM-Generated RTL. Consider a simple accumulation loop. An LLM-generated RTL typically captures functional correctness but leaves pipeline structure implicit, resulting in a single-cycle feedback path that limits achievable throughput. In contrast, a compiler-driven flow introduces explicit staging to enforce a target initiation interval, enabling one operation per cycle with bounded latency.

```
// LLM-generated RTL
always_ff @(posedge clk) begin
  acc <= acc + in;
end
```

```
// Compiler-generated RTL
always_ff @(posedge clk) begin
  acc_stage1 <= in;
  acc_stage2 <= acc_stage2 + acc_stage1;
end
```

This distinction explains the performance trends observed in our evaluation. Such gaps arise because LLMs generate RTL via autoregressive token prediction, where scheduling, resource allocation, and pipeline depth are inferred implicitly during generation rather than resolved through explicit intermediate representations. By contrast, compiler-driven frameworks resolve these decisions using explicit scheduling and optimization passes guided by cost models targeting throughput and hardware efficiency. While LLM-based workflows often rely on sampling multiple candidates and selecting those that pass verification, this strategy introduces variability and does not guarantee convergence toward high-performance implementations.

V. CONCLUSION

This paper presents a controlled empirical comparison of compiler-driven and LLM-driven SystemVerilog generation across a diverse set of kernels. Our results show that while LLM-based approaches enable rapid design exploration and achieve functional correctness for simple designs, compiler-driven flows consistently deliver up to $1.5\times$ higher throughput and 50% lower resource usage on complex, performance-critical workloads due to explicit scheduling and optimization. These findings indicate that LLMs are well-suited for early-stage RTL generation and prototyping, whereas compiler infrastructures remain essential for predictable, scalable, and performance-oriented hardware design projects.

REFERENCES

[1] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P.-l. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, J. Hu, R. Hull, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, T. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snelham, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, "In-datacenter performance analysis of a tensor processing unit," *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pp. 1–12, 2017.

[2] J. L. Hennessy and D. A. Patterson, "A new golden age for computer architecture," *Communications of the ACM*, vol. 62, no. 2, pp. 48–60, 2019.

[3] H. Chen, N. Zhang, S. Xiang, Z. Zeng, M. Dai, and Z. Zhang, "Allo: A programming model for composable accelerator design," *Proc. ACM Program. Lang.*, vol. 8, no. PLDI, Jun. 2024. [Online]. Available: https://doi.org/10.1145/3656401

[4] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, N. Vasilache, and O. Zinenko, "Mlir: Scaling compiler infrastructure for domain specific computation," *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pp. 2–14, 2021.

[5] A. Eldridge, Y. Tan, B. Tu, J. Wright, Z. Petersen, and Arvind, "Mlir-based hardware design with circt," in *Proceedings of the ACM/IEEE Workshop on Open-Source EDA Technology (WOSET)*, 2021.

[6] calyxir, "calyx: Intermediate language (il) for hardware accelerator generators," https://github.com/calyxir/calyx, 2026, gitHub repository (accessed February 2026).

[7] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, J. Carrasquilla, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, K. Knight, M. Brundage, M. Murati, K. Mayer, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[8] OpenAI, "Gpt-4 technical report," OpenAI, Tech. Rep., 2023.

[9] Y. Li, Y. Chen, J. Liu, and Z. Zhang, "Chipscribe: End-to-end chip design automation via large language models," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023.

[10] Y. Liu, H. Zhang, Y. Zhou, Z. Shi, C. Xu, and Q. Xu, "Deeprtl2: A versatile model for rtl-related tasks," 2025. [Online]. Available: https://arxiv.org/abs/2506.15697

[11] K. Majumder and U. Bondhugula, "Hir: An mlir-based intermediate representation for hardware accelerator description," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, ser. ASPLOS '23. New York, NY, USA: Association for Computing Machinery, 2024, p. 189–201. [Online]. Available: https://doi.org/10.1145/3623278.3624767

[12] N. B. Agostini, A. Limaye, M. Minutoli, V. G. Castellana, J. Manzano, A. Tumeo, S. Curzel, and F. Ferrandi, "Soda synthesizer: An open-source, multi-level, modular, extensible compiler from high-level frameworks to silicon," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, ser. ICCAD '22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: https://doi.org/10.1145/3508352.3561101

[13] W. S. Moses, L. Chelini, R. Zhao, and O. Zinenko, "Polygeist: Raising c to polyhedral mlir," in *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE, 2021, pp. 45–59.

[14] F. Ferrandi, V. G. Castellana, S. Curzel, P. Fezzardi, M. Fiorito, M. Lattuada, M. Minutoli, C. Pilato, and A. Tumeo, "Bambu: an open-source research framework for the high-level synthesis of complex applications," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 1327–1330.

[15] S. Basalama and J. Cong, "Stream-hls: Towards automatic dataflow acceleration," in *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2025, pp. 103–114.

[16] R. Nigam, S. Thomas, Z. Li, and A. Sampson, "A compiler infrastructure for accelerator generators," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 804–817.

[17] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "Verigen: A large language model for verilog code generation," 2023.

[18] J. Tang, J. Qin, K. Thorat, C. Zhu-Tian, Y. Cao, Y. Zhao, and C. Ding, "Hivegen – hierarchical llm-based verilog generation for scalable chip design," 2024.

[19] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "Invited paper: Verilogeval: Evaluating large language models for verilog code generation," in *2023 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023, pp. 1–8.

[20] H. Huang, Z. Lin, Z. Wang, X. Chen, K. Ding, and J. Zhao, "Towards llm-powered verilog rtl assistant: Self-verification and self-correction," 2024.

[21] Y. Zhao, D. Huang, C. Li, P. Jin, Z. Nan, T. Ma, L. Qi, Y. Pan, Z. Zhang, R. Zhang, X. Zhang, Z. Du, Q. Guo, X. Hu, and Y. Chen, "Codev: Empowering llms for verilog generation through multi-level summarization," 2024.

[22] J. Zuo, J. Liu, X. Wang, Y. Liu, N. Goli, T. Xu, H. Zhang, U. R. Tida, Z. Jia, and M. Zhao, "Complexvcoder: An llm-driven framework for systematic generation of complex verilog code," 2025.

[23] W.-P. Hsin, R.-H. Deng, Y.-T. Hsieh, E.-M. Huang, and S.-H. Hung, "Evolve: Evolutionary search for llm-based verilog generation and optimization," 2026.

[24] OpenAI, "Introducing gpt-5.2," https://openai.com/index/introducing-gpt-5-2/, December 2025, accessed: 2026-02-08.

[25] G. DeepMind, "Gemini 3: State-of-the-art multimodal reasoning and intelligence," Google DeepMind, Technical Report, November 2025. [Online]. Available: https://deepmind.google/technologies/gemini/